```
# taken from vist-diffusion-pinn-5200-half-channel-plus-units-load-skip-5-cells.
py

import numpy as np
from numpy import linalg as LA
import math as m
import os
import sys
from matplotlib.image import imread
import matplotlib.pyplot as plt
from matplotlib import rcParams # for changing default values
import scipy.io as sio
from scipy.optimize import minimize
import timeit
import torch
import random
import torch.nn as nn
import torch.optim as optim
from scipy.integrate import odeint
from torch.autograd import grad
import torch.optim.lr_scheduler as lr_scheduler

plt.close('all')
plt.interactive(True)
# set all fontsizes
rcParams["font.size"] = 16

viscos = 1/5200

# load DNS data
DNS_mean=np.genfromtxt("LM_Channel_5200_mean_prof.dat",comments="%")
y_DNS=DNS_mean[:,0];
yplus_DNS=DNS_mean[:,1];
u_DNS=DNS_mean[:,2];
dudy_DNS=np.gradient(u_DNS,yplus_DNS)

DNS_stress=np.genfromtxt("LM_Channel_5200_vel_fluc_prof.dat",comments="%")
u2_DNS=DNS_stress[:,2];
v2_DNS=DNS_stress[:,3];
w2_DNS=DNS_stress[:,4];
uv_DNS=DNS_stress[:,5];
k_DNS=0.5*(u2_DNS+v2_DNS+w2_DNS)
dkdy_DNS=np.gradient(k_DNS,yplus_DNS,edge_order=2)
d2kdy2_DNS=np.gradient(dkdy_DNS,yplus_DNS,edge_order=2)

#y/delta y^+ Prod Turb_Transport Viscous_Transport Pressure_Strain Pressure_Tran
sport Viscous_Dissipation           Balance
DNS_k_terms=np.genfromtxt("LM_Channel_5200_RSTE_k_prof.dat",comments="%")

diss_DNS=DNS_k_terms[:,7]
Pk_DNS=DNS_k_terms[:,2]
diff_DNS=DNS_k_terms[:,3]
diff_DNS_visc =   DNS_k_terms[:,4]

diss_iso_DNS = np.maximum(diss_DNS-diff_DNS_visc,0)

diss_DNS=diss_DNS
Pk_DNS=Pk_DNS
diff_DNS=diff_DNS
diss_iso_DNS=diss_iso_DNS
diff_DNS_visc = diff_DNS_visc
```



```
vist_DNS = np.abs(uv_DNS/dudy_DNS)

# load k-omega grid
kom_data = np.loadtxt('y_u_k_om_uv_5200-RANS-half-channel.txt')
y_kom = kom_data[:,0]
k_kom = kom_data[:,2]
om_kom = kom_data[:,3]
vist_kom = k_kom/om_kom/viscos

# skip 5 cells near the wall
j=5
y_kom = y_kom[j:]
vist_kom = vist_kom[j:]

nj = len(y_kom)

viscos_lam = np.ones(nj)

k_DNS = np.interp(y_kom, y_DNS, k_DNS)
k_DNS = torch.tensor(k_DNS, requires_grad=True, dtype=torch.float32).view((-1, 1
))

Pk_DNS = np.interp(y_kom, y_DNS, Pk_DNS)
Pk_DNS = torch.tensor(Pk_DNS, requires_grad=True, dtype=torch.float32).view((-1,
 1))

diss_DNS = np.interp(y_kom, y_DNS, diss_DNS)
diss_DNS = torch.tensor(diss_DNS, requires_grad=True, dtype=torch.float32).view(
(-1, 1))

d2kdy2_DNS = np.interp(y_kom, y_DNS, d2kdy2_DNS)
d2kdy2_DNS = torch.tensor(d2kdy2_DNS, requires_grad=True, dtype=torch.float32).v
iew((-1, 1))

dkdy_DNS = np.interp(y_kom, y_DNS, dkdy_DNS)
dkdy_DNS = torch.tensor(dkdy_DNS, requires_grad=True, dtype=torch.float32).view(
(-1, 1))

diff_DNS_visc = np.interp(y_kom, y_DNS, diff_DNS_visc)

diff_DNS = np.interp(y_kom, y_DNS, diff_DNS)
diff_DNS = torch.tensor(diff_DNS, requires_grad=True, dtype=torch.float32).view(
(-1, 1))

vist_DNS = np.interp(y_kom, y_DNS, vist_DNS)
vist_DNS_np = vist_DNS
vist_DNS = torch.tensor(vist_DNS, requires_grad=True, dtype=torch.float32).view(
(-1, 1))

viscos_lam = torch.tensor(viscos_lam, requires_grad=True, dtype=torch.float32).v
iew((-1, 1))

yplus_DNS = np.interp(y_kom, y_DNS, yplus_DNS)
yplus_DNS_np = yplus_DNS
yplus_DNS = torch.tensor(yplus_DNS, requires_grad=True, dtype=torch.float32).vie
w((-1, 1))

y_DNS = torch.tensor(y_kom, requires_grad=True, dtype=torch.float32).view((-1, 1
))

# b.c.
vist_0 = vist_DNS[0]
```

```
vist_1 = vist_DNS[-1]

y = yplus_DNS

# Define get_derivative
dtype = torch.float
device = torch.device("cpu")

def get_derivative(f, y):
    """Compute the derivative of f = f(y) with respect to y."""
    df_dy = grad(f, y, torch.ones(y.size()[0], 1, device=device), create_graph=T
rue)[0]
    return df_dy

class MyNet2(nn.Module):
  def __init__(self):
    super().__init__()
    self.ll1 = nn.Linear(in_features=1,out_features=10)
    self.tanh = nn.Tanh()
    self.ll2 = nn.Linear(in_features=10,out_features=10)
    self.ll3 = nn.Linear(in_features=10,out_features=10)
    self.output = nn.Linear(in_features=10,out_features=1)

  def forward(self,x):
        out = self.ll1(x)
        out = self.tanh(out)
        out = self.ll2(out)
        out = self.tanh(out)
        out = self.ll3(out)
        out = self.output(out)
        return out

# Create an instance
model = MyNet2()

#%% Define loss function
def PDE(y, vist_pred):
        """Compute the cost function."""
        global temp
        # Differential equation loss
        dvist_dy = get_derivative( vist_pred,y)
        temp = (vist_pred+viscos_lam) * d2kdy2_DNS + dkdy_DNS*dvist_dy

        boundary_condition_loss = 0
        differential_equation_loss = temp  + (Pk_DNS - diss_DNS)
        imbalance = differential_equation_loss
        differential_equation_loss = torch.sum(differential_equation_loss ** 2)
        # Boundary condition loss initialization
        boundary_condition_loss = 0
        # Sum over dirichlet boundary condition losses
        boundary_condition_loss += (vist_pred[0] - vist_0) ** 2
        boundary_condition_loss += (vist_pred[-1] - vist_1) ** 2

        return differential_equation_loss, boundary_condition_loss, imbalance

def loss_and_PDE(y_tensor):
    optimizer.zero_grad() # Clear gradients from the previous iteration
    outputs = model(y_tensor)  #get vist_t
    loss_de,loss_bc, imbalance = PDE(y_tensor, outputs) # Compute the loss
    loss = loss_de+1000.*loss_bc
# Calculate the L1 regularization term
    l1_regularization = torch.tensor(0.)
```



```
    for param in model.parameters():
        l1_regularization += torch.norm(param, p=1)

    # Add the L1 regularization term to the loss
    lambda_l1=0.
    loss += lambda_l1 * l1_regularization # Compute the loss
    loss.backward() # Compute gradients using backpropagation
    return loss,loss_de,loss_bc, imbalance

#%% training
prev_loss = float('inf')  # Initialize with a large value
max_no_epoch=100000
#max_no_epoch=2

optim_alg='Adam'
#optim_alg='SGD'
learning_rate = 0.2  #  4221.8496 milestones=[500000]
learning_rate = 0.2  #  0.6554632 milestones=[6400,43000,54578]
print('Adam taken')
optimizer = optim.Adam(model.parameters(), lr=learning_rate)

#for saving training result
differential_equation_loss_history = np.zeros(max_no_epoch)
boundary_condition_loss_history = np.zeros(max_no_epoch)
loss_min = 1e30
# Training loop
for epoch in range(max_no_epoch):
    loss,loss_de,loss_bc, imbalance = loss_and_PDE(y)
    differential_equation_loss_history[epoch] += loss_de
    boundary_condition_loss_history[epoch] += loss_bc
    optimizer.step()
    loss_change = prev_loss - loss
    prev_loss = loss

# Define checkpoint
    if epoch == 0:
        checkpoint = torch.load('checkpoint-vist-5200-plus-units-save-5-cells.ct')

# Apply the state_dict to model and optimizer
        model = MyNet2()  # Initialize model; Ensure it's the same architecture
        model.load_state_dict(checkpoint['model_state_dict'])

        optimizer = optim.Adam(model.parameters(), lr=learning_rate) # Initialize
 optimizer; Ensure it's the same optimizer type
        optimizer.load_state_dict(checkpoint['optimizer_state_dict'])
        scheduler = optim.lr_scheduler.MultiStepLR(optimizer,  milestones=[6400,1
6700,19200,37500,38000,80000,94000], gamma=0.5)

# Retrieve the training epoch
        epoch = checkpoint['epoch']
        loss = checkpoint['loss']

        model.train()  # For training mode (resuming training)

    scheduler.step()

    loss_np = loss.detach().numpy()

# Print the loss every epoch
    loss_min = np.minimum(loss_np,loss_min)
    loss_min = np.float32(loss_min)
    torch.set_printoptions(precision=4)
```

```
    print(f"Epoch {epoch+1}, Learning Rate: {scheduler.get_last_lr()[0]}, Loss: {loss_np[0]:.2e}, Loss_min: {l
oss_min[0]:.2e}")
    if loss_np < 5e-5:
        vist_pred = model(y)
        vist_pred_np =  vist_pred.detach().numpy()[:,0]

        print('break')

        break

# Plot loss_function
fig, ax = plt.subplots(nrows=1, ncols=1) # Create a figure with one subplot
ax.semilogy(np.arange(len(boundary_condition_loss_history)), boundary_condition_
loss_history,color='red', label='bc error')
ax.semilogy(np.arange(len(boundary_condition_loss_history)), differential_equati
on_loss_history,color='blue',label="diff eq error")
plt.xlabel(r'epochs')
ax.set_title(r'Errors')
ax.grid(visible=True)
ax.legend(loc='best')
plt.savefig('loss-half-5200-plus-units-load-5-cells.png',bbox_inches='tight')


######################## plot vist zoom
fig, ax = plt.subplots(nrows=1, ncols=1) # Create a figure with one subplot
plt.subplots_adjust(left=0.20,bottom=0.20)
ax.plot(yplus_DNS_np, vist_DNS_np,color='r',linestyle=':',linewidth=5, label='DN
S')
vist_pred = model(y)
vist_pred_np =  vist_pred.detach().numpy()[:,0]
ax.plot(yplus_DNS_np, vist_pred_np,color='k',linestyle='-',linewidth=2, label=r"
$\nu_{t,\mathrm{pred}}$")
ax.plot(y_kom/viscos, vist_kom,color='r',linestyle='-',linewidth=2, label=r"$\nu_
{t,k-\omega}$")
ax.legend(loc='best')
plt.xlabel('$y^+$')
plt.ylabel(r'$\nu_t/\nu$')
plt.xlim(0,100)
ax.axis([0,50,0,20])
ax.grid(visible=True)
plt.savefig('vist-PINN-5200-plus-units-load-5-cells-zoom.png',bbox_inches='tight')

np.savetxt('vist_pred-PINN-from-vist-diffusion-pinn-5200-plus-units-load-5-cells.txt',vist_pred_np
)

######################## plot diffusion term
fig, ax = plt.subplots(nrows=1, ncols=1) # Create a figure with one subplot
plt.subplots_adjust(left=0.20,bottom=0.20)
dkdy_DNS_np =  dkdy_DNS.detach().numpy()[:,0]
diff_DNS =  diff_DNS.detach().numpy()[:,0]
y_DNS =  y_DNS.detach().numpy()[:,0]
term = dkdy_DNS_np*vist_pred_np
dvist_dy = get_derivative( vist_pred,y)
diff_non_conserv = vist_pred * d2kdy2_DNS + dkdy_DNS*dvist_dy
diff_DNS_pred = np.gradient(term,y_DNS)
plt.plot(y_DNS/viscos, (diff_DNS_pred+diff_DNS_visc),color='k',linestyle='-',lin
ewidth=2, label=r"predicted")
plt.plot(y_DNS/viscos, (diff_DNS+diff_DNS_visc),color='b',linestyle='-',linewidt
h=2, label=r"DNS")
plt.plot(y_DNS/viscos, diff_non_conserv.detach().numpy(),color='r',linestyle='-'
,linewidth=2, label=r"non-cons")
ax.legend(loc='best')
```



```
plt.xlabel('$y^+$')
plt.ylabel('diffusion')
ax.grid(visible=True)
plt.savefig('diffusion-PINN-5200-plus-units-load-5-cells.png',bbox_inches='tight')


######################## plot diffusion term zoom more
fig, ax = plt.subplots(nrows=1, ncols=1) # Create a figure with one subplot
plt.subplots_adjust(left=0.20,bottom=0.20)
plt.plot(y_DNS/viscos, diff_DNS,'r--',linewidth=2, label=r"DNS")
plt.plot(y_DNS/viscos, diff_non_conserv.detach().numpy(),'b-',linewidth=2, label
=r"PINN")
plt.plot(y_DNS/viscos, diff_non_conserv.detach().numpy(),'bo',linewidth=2)
ax.legend(loc='best')
plt.xlabel('$y^+$')
plt.ylabel('diffusion')
plt.xlim(0,100)
ax.grid(visible=True)
plt.savefig('diffusion-PINN-5200-plus-units-load-5-cells-zoom-more.png',bbox_inches='tight')


############################### Plot imbalance, Pk and diss zoom
fig, ax = plt.subplots(nrows=1, ncols=1) # Create a figure with one subplot
plt.subplots_adjust(left=0.20,bottom=0.20)
ax.plot(yplus_DNS.detach().numpy(), imbalance.detach().numpy(),color='r',linesty
le=':',linewidth=5, label='imbalance')
ax.plot(y.detach().numpy(), Pk_DNS.detach().numpy(),color='k',linestyle='-',line
width=2, label=r"$P_{k,DNS}$")
ax.plot(y.detach().numpy(), -diss_DNS.detach().numpy(),color='b',linestyle='-',l
inewidth=2, label=r"$\varepsilon_{DNS}$")
ax.plot(y.detach().numpy(), diff_DNS_visc,color='r',linestyle='--',linewidth=2,
label=r"$D^\nu_{DNS}$")
ax.plot(y.detach().numpy(), diff_DNS,color='b',linestyle='--',linewidth=2, label
=r"$D^t_{DNS}$")
ax.legend(loc='best')
plt.xlabel('$y^+$')
ax.grid(visible=True)
plt.xlim(0,100)
plt.savefig('k-balance-PINN-5200-plus-units-load-5-cells-zoom.png',bbox_inches='tight')
```